

Product Requirements Specification (PRS)

1. Product Vision & Intent

Businesses struggle with slow decision-making, manual data analysis, and lack of future insights.

Instead of directly building an AI system, first identify where decisions are delayed or inefficient.

Ask: *What problems do decision-makers face?*

- Time-consuming report generation
- Inaccurate or outdated data
- No predictive insights (only past data available)
- Difficulty in identifying trends or anomalies

Clearly define what improvement your system will bring:

- Reduce analysis time from hours to minutes
- Increase prediction accuracy (e.g., 85%+ forecasting accuracy)
- Provide real-time automated insights
- Detect anomalies instantly

Guiding Questions

Business managers, data analysts, and decision-makers who need quick and accurate insights from data.

Data analysis is mostly manual, takes more time, and decisions are based only on past data. There is no proper prediction, and important trends or problems may be missed.

The system will be successful if it provides fast insights, accurate predictions, reduces manual work, and helps users make better decisions in real time.

Example

- **Problem:** Users struggle to track academic progress because data is scattered across multiple tools, making analysis difficult and time-consuming.
- **Success:** Within 45 days, at least 80% of users will regularly (weekly) track their progress using the system with improved ease and accuracy.

2. Scope Definition

2.1 In-Scope Capabilities

Describe what the product enables, not screens or pages

Focus on what users can *do* with the system, such as analyzing data, predicting outcomes, and making decisions—rather than describing dashboards or UI.

Write as verbs + nouns (capabilities)

Express features as actions:

- Analyze business data
- Predict future trends
- Detect anomalies
- Generate automated insights
- Support decision-making

Guiding Questions

The system must be able to collect and analyze data automatically, generate insights, predict future trends, and support quick decision-making. It should reduce manual effort and provide accurate, real-time information.

-
- Collect and integrate data from sources
 - Analyze data and generate basic insights
 - Provide simple dashboards/reports
 - Perform basic predictions (e.g., trends)

Example

- Manage user profiles
Store and update user information to personalize insights and track individual performance.
- Track progress against milestones
Monitor user progress over time and compare it with predefined goals or targets.
- Generate performance summaries
Automatically create reports that highlight key metrics, trends, and overall performance.

2.2 Out-of-Scope Capabilities

- Explicitly defer or postpone non-essential features
Focus only on what delivers core value. Avoid adding complex features in the beginning.
- Postpone features that require major effort
Advanced functionalities like complex AI models, full automation, or integrations can be added later.
- Avoid non-critical features
Features that do not directly help in data analysis, prediction, or decision-making should be delayed.

Example

- **Mobile native apps**
Instead of building Android/iOS apps initially, the system can start with a **web-based dashboard**. Mobile apps can be developed later after the core system is stable.
- **Third-party marketplace integrations**
Integrations with external platforms (like CRM, ERP, or marketplaces) can be postponed. Initially, the system can work with **limited or internal data sources**.

3. Functional Requirements

User Authentication & Profile Management

- User Actions:
 - Register a new account
 - Log in / log out
 - Update profile details
 - System Responses:
 - Validate credentials
 - Store and retrieve user data securely
 - Provide access to personalized dashboard
-

2. Data Ingestion & Integration

- User Actions:
 - Upload datasets (CSV, Excel, etc.)
 - Connect external data sources (APIs, databases)
 - System Responses:
 - Validate and clean incoming data
 - Store data in structured format
 - Handle errors for invalid or incomplete data
-

3. Data Analysis & Processing

- User Actions:
 - Select dataset for analysis

- Choose analysis type (trend, comparison, etc.)
 - System Responses:
 - Process data using AI/ML algorithms
 - Generate insights and summaries
 - Highlight key patterns and anomalies
-

4. Predictive Analytics

- User Actions:
 - Request future predictions
 - Select prediction parameters
 - System Responses:
 - Apply trained machine learning models
 - Generate forecasts and probability scores
 - Display prediction results with confidence levels
-

5. Dashboard & Visualization

- User Actions:
 - View reports and dashboards
 - Customize charts and filters
 - System Responses:
 - Display interactive visualizations (graphs, charts)
 - Update visuals in real-time based on filters
 - Provide export options (PDF, Excel)
-

6. Alerts & Recommendations

- User Actions:
 - Set alert conditions
 - Subscribe to notifications
- System Responses:

- Monitor data continuously
 - Trigger alerts for anomalies or thresholds
 - Provide AI-based recommendations
-

7. System Administration

- User Actions:
 - Manage users and permissions
 - Configure system settings
- System Responses:
 - Enforce role-based access control
 - Maintain system logs and audit trails

3.2 Functional Workflows

Walk through complete end-to-end user journeys

- Consider both normal flow and edge cases
- Include failure scenarios and alternate path

Example Workflows

1. User Login Workflow

Steps:

1. User enters login credentials
2. System validates credentials
3. User is redirected to dashboard

Failure/Alternate Paths:

- Invalid credentials → Error message displayed
 - Account locked → Prompt to reset password
 - Server issue → Retry option shown
-

2. Data Upload Workflow

Steps:

1. User uploads dataset
2. System validates file format and structure
3. Data is stored in database
4. Confirmation message displayed

Failure/Alternate Paths:

- Unsupported file format → Error message
 - Missing or corrupted data → Request re-upload
 - Upload failure → Retry option
-

3. Data Analysis Workflow

Steps:

1. User selects dataset
2. User chooses analysis type
3. System processes data
4. Results are displayed on dashboard

Failure/Alternate Paths:

- No data available → Prompt to upload data
 - Processing error → Display error with retry option
 - Timeout → Notify user and allow reprocessing
-

4. Predictive Analytics Workflow

Steps:

1. User selects prediction model
2. System applies trained ML model
3. Forecast results are generated
4. Results are visualized

Failure/Alternate Paths:

- Insufficient data → Warning message
 - Model failure → Switch to fallback model
 - Invalid input → Request correct parameters
-

5. Alert & Notification Workflow

Steps:

1. User sets alert conditions
2. System monitors data continuously
3. Alert is triggered when condition is met
4. Notification sent to user

Failure/Alternate Paths:

- Notification failure → Retry or log issue
- Incorrect threshold → Allow user to update settings

Example User Workflow

1. User logs in
2. User selects course
3. System loads progress data
4. System displays recommendations
5. Error shown if data unavailable

3.3 Business Rules & Logic

How to Identify

- Walk through end-to-end user journeys
Describe step-by-step how a user interacts with the system.
- Include failure and alternate paths
Consider what happens if something goes wrong.

◆ Example Workflow (For Your System)

1. User logs into the system
 2. User uploads/selects data source
 3. System processes and analyzes data
 4. System generates insights and predictions
 5. Results are displayed on dashboard
 6. Error message shown if data is missing or invalid
-

◆ Business Rules & Logic

- Look for decisions and validations
Identify conditions that control system behavior.
 - Capture “if-then” logic
Define rules clearly for system actions.
-

◆ Examples (For Your System)

- Data is processed only if it is valid and complete
- Predictions are generated only if sufficient data is available
- Progress cannot exceed 100%
- Alerts are triggered if unusual patterns are detected

3.4 Functional Constraints & Limits

- **Identify boundaries for performance and usability**
Set limits to ensure the system runs smoothly and avoids overload.
 - **Think in terms of maximums**
Define upper limits for users, data, and operations.
-

◆ Guiding Questions

- → Restrict excessive usage (e.g., too many requests)
 - → Limit data size, number of users, or operations
-

◆ Examples (For Your System)

- Maximum number of concurrent users supported at a time
- Limit on data size uploaded for analysis (e.g., 10–50 MB)
- Maximum number of reports generated per day
- Limited number of active sessions per user

3.5 Functional Acceptance Criteria

- **Define observable outcomes**
Clearly state what should happen when a feature works correctly.
 - **Write testable conditions**
Ensure conditions are measurable and can be verified by testers.
-

◆ Guiding Question

- → The system should perform the expected function correctly and consistently.
-

◆ Examples (For Your System)

- Given valid data, insights are generated and displayed within a few seconds
- Predictions are calculated and shown accurately for input data
- User progress updates are saved and reflected immediately
- Reports are generated without errors

4. Non-Functional Requirements

4.1 Performance

- Focus on user perception.
- Use percentiles instead of averages.

Guiding Questions

- How fast is “fast enough”?
- What delays break user trust?

Example

- For 95% of requests respond within 1.5 seconds.

4.2 Scalability

- **Project realistic growth scenarios**
Think about how the system will handle more users, data, and workload in the future.
 - **Use ranges instead of exact numbers**
When unsure, define approximate growth (e.g., thousands to tens of thousands of users).
-

◆ Guiding Questions

- → System should handle increased data and users without slowing down
 - → Core architecture and data processing pipeline
-

◆ Example (For Your System)

- System supports growth from **1,000 to 50,000 users** without major architectural changes
- Can handle increasing data volume and real-time processing
- Scales by adding more servers or cloud resources

4.3 Availability & Reliability

- Consider impact of downtime
Understand how system unavailability affects business decisions and operations.
- Define acceptable failure behavior
Specify how the system should behave during failures and how it recovers.

◆ Guiding Questions

- → Yes, by showing last available data instead of crashing
- → Within seconds or minutes to avoid disruption

◆ Examples (For Your System)

- 99.5% uptime maintained every month
- Automatic recovery from system or server failures
- Backup systems to ensure data is not lost
- Graceful failure (system continues with limited functionality)

- Identify sensitive data
Recognize data that needs protection (user data, business data, predictions).
- Apply least-privilege access
Give users only the access they need—nothing more.

◆ Guiding Questions

- → User details, business insights, reports
 - → Define roles (admin, analyst, user)
 - → Restrict unnecessary access
-

◆ Examples (For Your System)

- All data encrypted at rest and in transit
- Role-based access control (RBAC) enforced
- Secure authentication and login system
- Regular monitoring for security threats

4.5 Extensibility & Maintainability

- **Anticipate change, not features**
Design the system keeping future updates in mind (new data sources, improved models).
 - **Focus on modularity**
Build the system in separate modules so changes can be made without affecting the whole system.
-

◆ Guiding Questions

- → AI models, data sources, user requirements
 - → New analytics features, prediction models, integrations
-

◆ Example (For Your System)

- New modules (e.g., advanced prediction models) can be added without modifying core services
- Easy integration of new data sources or APIs
- System updates without downtime

5. Data & Integration Requirements

- ◆ Identify nouns (entities) in workflows

Determine key data elements involved in the system.

- Map data ownership and flow
Understand where data comes from, how it moves, and who controls it.
-

◆ Examples (For Your System)

- Entities: User, Data Source, Report, Prediction, Insight, Progress

- Data Flow: Data collected → processed → analyzed → insights generated
- Ownership: Business/user owns input data; system processes and generates outputs
-

6. UX, Accessibility & Interaction Principles

Specify requirements like device compatibility and accessibility instead of detailed UI designs.

- Focus on usability standards
Ensure the system is easy to use, intuitive, and accessible for all types of users.

◆ Examples (For Your System)

- Must be usable on desktop and tablet devices
- Simple and intuitive dashboard for non-technical users
- Fast and responsive interface
- Accessible design following standards (e.g., WCAG 2.1 AA)

7. Observability, Metrics & Monitoring

Decide what to track in real usage

Identify what you need to monitor when users interact with the system (usage, performance, errors).

- Tie metrics to success criteria
Measure things that directly show whether the system is achieving its goals (speed, accuracy, adoption).

◆ Examples (For Your System)

- Number of active users using the system daily/weekly
- Accuracy of predictions (e.g., forecasting accuracy %)
- Time taken to generate insights (latency)

- Error rate in data processing (< 1%)
- Alerts triggered for anomalies or system issues

8. Risks, Assumptions & Dependencies

- Be explicit about uncertainty
Identify areas where outcomes are unclear, such as new AI models or data availability.
- Check technology stability
Ensure the tools, platforms, and algorithms you plan to use are reliable.
- Capture assumptions (what you're betting on)
Clearly state expectations like data quality, user adoption, or system performance.

9. Validation & Release Criteria

Clearly list the conditions that must be met before the system is considered ready.

Ensure the system not only works correctly (functions) but also performs well (speed, security, reliability).

-
-  **Example (For Your System)**
 - All core features (data analysis, prediction, reporting) are working correctly
 - Users can successfully track progress and view insights
 - System generates accurate performance summaries
 - Performance is fast (real-time or near real-time results)
 - Security measures are implemented (data protection)